

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO4 ASSESSMENT TOOL 2 – CI/CD Pipeline Design Exercise

Course Code:	CSA10 / CSA1003	Course Title:	Software Engineering
CO Assessed:	CO4	Assessment:	Assessment Tool 2 – CI/CD Pipeline Design Exercise
Weightage:	20%	Total Marks:	25
CO4 Statement:	Implement robust testing, quality assurance, and DevOps practices, emphasizing security, reliability, and ethics		
Student Name:	K Madan Mohan Reddy	Reg. No.:	192411206
Date:	22/08/2026	Duration:	60 minutes

Code of Conduct

I K Madan Mohan Reddy (192411206) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____ Byreddy _____

Annexure A – CI/CD Pipeline Design Exercise

“Online Loan Eligibility Prediction and Approval Management Platform”

The exercise should include:

Analyze the Project Requirements

1.1 Identify the CI/CD Needs of the Assigned Problem Statement

The Online Loan Eligibility Prediction and Approval Management Platform contains multiple components such as the frontend, backend APIs, AI prediction model, database, and loan application processing. Because these components require regular updates, a CI/CD pipeline can automate the process of building, testing, and deploying the application. Whenever developers make changes to the source code, the pipeline should automatically verify whether the new changes work correctly. Automated testing is important because incorrect code can affect loan eligibility calculations, AI predictions, applicant data processing, and approval decisions. Code quality and security checks should also be performed before deployment. The system requires separate development, testing, and production environments to reduce deployment risks. Docker can be used to provide a consistent application environment. Automated deployment helps reduce manual errors and saves development time. Rollback functionality is also required so that the previous stable version can be restored if a deployment fails. Overall, CI/CD provides a faster, safer, and more reliable development and deployment process.

.Main CI/CD Needs

- Automatic source code integration.
 - Automated application building.
 - Automated unit and integration testing.
 - AI model validation.
 - Code quality and security checking.
 - Docker image creation.
 - Deployment to testing/staging environment.
 - Production deployment after approval.
 - Automatic notifications.
 - Failure detection and rollback..
-

2. Design the CI/CD Pipeline Workflow

2.1 CI/CD Pipeline Stages

The proposed CI/CD pipeline begins when a developer commits code to the Git repository. The source code is automatically checked by the CI/CD system. The application is then built and its dependencies are installed. Automated unit, API, integration, and AI model tests are executed to identify errors. Code quality and security analysis are performed before creating the deployment package. After successful validation, a Docker image is created and stored in a container registry. The application is then deployed to the testing or staging environment. After successful staging validation and approval, the application is deployed to the production environment. Monitoring is performed after deployment to identify failures. If a serious problem occurs, the pipeline can automatically or manually roll back to the previous stable version.

CI/CD Workflow

Developer



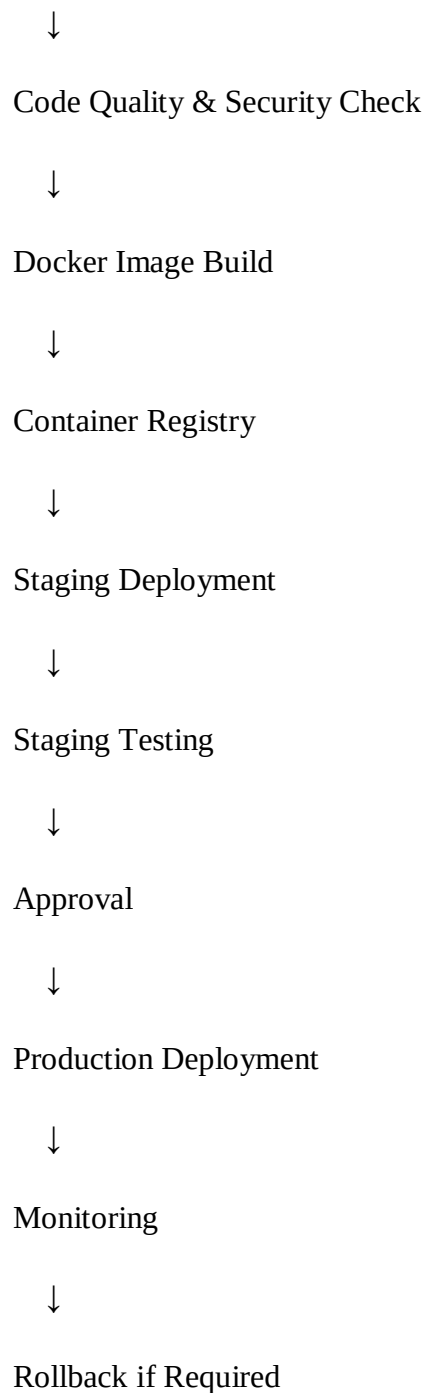
Git Repository



Build



Automated Testing



3. Identify Suitable Tools and Technologies

3.1 Tools for Each CI/CD Stage

Different tools can be selected based on the requirements of the Online Loan Eligibility Prediction and Approval Management Platform. Git and GitHub can be used for source code management because they provide version control, branching, collaboration, and repository management. GitHub Actions can

automate the CI/CD workflow by integrating directly with the GitHub repository. Python and pip can be used to install backend and AI dependencies. Pytest can be used for automated unit and integration testing. SonarQube can be used for code quality analysis and identifying code issues. Docker can package the application and its dependencies into containers. A container registry can store the generated Docker images. The application can then be deployed to a cloud or server environment. These tools provide an automated workflow from code development to deployment.

Pipeline Stage	Suggested Tool	Purpose
Source Control	Git + GitHub	Manage source code and versions
CI/CD	GitHub Actions	Automate pipeline
Build	Python + pip	Install dependencies and build application
Testing	Pytest	Execute automated tests
Code Quality	SonarQube	Analyze code quality
Security	Dependency/SAST scanner	Identify vulnerabilities
Packaging	Docker	Create application containers
Registry	Container Registry	Store Docker images
Deployment	Cloud/Server + Docker	Deploy application
Notification	GitHub/Email/Teams	Report pipeline status

3.2 Justification for Tool Selection

GitHub Actions is suitable for the Online Loan Eligibility Prediction and Approval Management Platform because it can automatically start the pipeline whenever code is pushed or a pull request is created. Pytest is appropriate for testing the Python backend and AI-based loan eligibility functions. Docker provides a consistent environment between development, testing, and production. SonarQube helps identify code quality and security problems before deployment. Git provides reliable version control and allows developers to maintain different branches for different features. Together, these tools reduce manual work and improve the reliability, security, and efficiency of the deployment process.

4. Define Automated Testing Strategies

4.1 Testing Strategy for the CI/CD Pipeline

Automated testing should be included at different stages of the pipeline so that errors are detected as early as possible. Unit testing verifies individual functions such as loan eligibility calculation, applicant data validation, and AI prediction functions. Integration testing checks communication between the backend, database, AI module, and APIs. API testing verifies whether endpoints correctly receive and return loan application data. AI model testing checks whether the prediction model produces accurate and acceptable

results using known test datasets. Regression testing ensures that new changes do not break existing functionality. Security testing can identify vulnerable dependencies and unsafe code. Performance testing can be performed for important APIs to check response time. Only when the required tests pass should the application proceed to the deployment stage.

Automated Testing Types

- Unit Testing: Tests individual functions and modules.
 - Integration Testing: Tests communication between system components.
 - API Testing: Verifies backend API requests and responses.
 - AI Model Testing: Checks prediction accuracy and expected output.
 - Regression Testing: Ensures existing features continue to work.
 - Security Testing: Identifies vulnerabilities and insecure dependencies.
 - Performance Testing: Checks response time and system behavior under load.
-

5. Design the Deployment Strategy

5.1 Development Environment

The development environment is used by developers to implement and test new features. Developers work on separate Git feature branches and make changes to the application. Local Docker containers can be used to maintain the same environment for all developers. Basic automated tests are executed before changes are pushed to the remote repository. This environment allows developers to identify and correct problems before the code reaches the testing environment.

5.2 Testing/Staging Environment

The staging environment provides an environment similar to production where the complete application can be tested before release. After successful CI checks, the Docker image is deployed to staging. Integration testing, API testing, database testing, and user acceptance testing can be performed in this environment. The AI prediction module can also be tested using representative environmental datasets. Any serious error found in staging should stop production deployment until it is corrected.

5.3 Production Environment

The production environment contains the stable version used by actual users. Only successfully tested and approved builds should be deployed to production. The production environment should use secure configuration, protected credentials, database backups, monitoring, and restricted access. Deployment can be performed using Docker containers and cloud infrastructure. After deployment, application health and system performance should be monitored continuously. If the new version causes serious problems, the previous stable version should be restored.

Deployment Flow

Development



Automated CI Tests



Staging Environment



Acceptance & Integration Testing



Approval



Production



Monitoring

6. Incorporate Security, Quality Checks, Notifications, and Rollback Mechanisms

6.1 Security and Quality Checks

Security should be integrated throughout the CI/CD pipeline rather than being performed only after deployment. Source code should be checked for common security problems and vulnerable dependencies. Passwords, database credentials, API keys, and other sensitive information should not be stored directly in the Git repository. Environment variables or secure repository secrets should be used instead. Code quality tools should check for duplicated code, code smells, and maintainability issues. Docker images should also be scanned for known vulnerabilities before deployment. Access to the production environment should be restricted to authorized users. These checks help prevent insecure or poor-quality code from reaching production.

6.2 Notifications

The pipeline should provide notifications whenever a build, test, or deployment succeeds or fails. Developers can receive notifications through email, Microsoft Teams, Slack, or GitHub notifications. A failed test should clearly identify the pipeline stage that caused the failure. Successful production deployment can also generate a notification to the development team. This allows developers to respond quickly to errors and reduces the time required to identify problems.

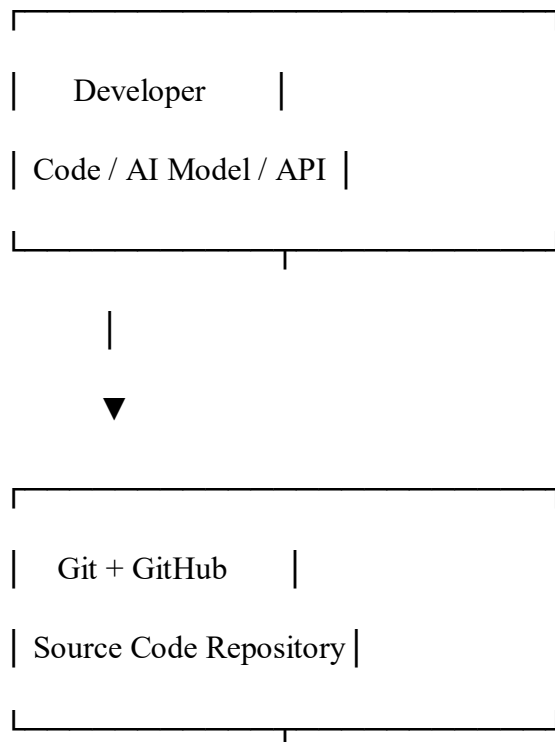
6.3 Rollback Mechanism

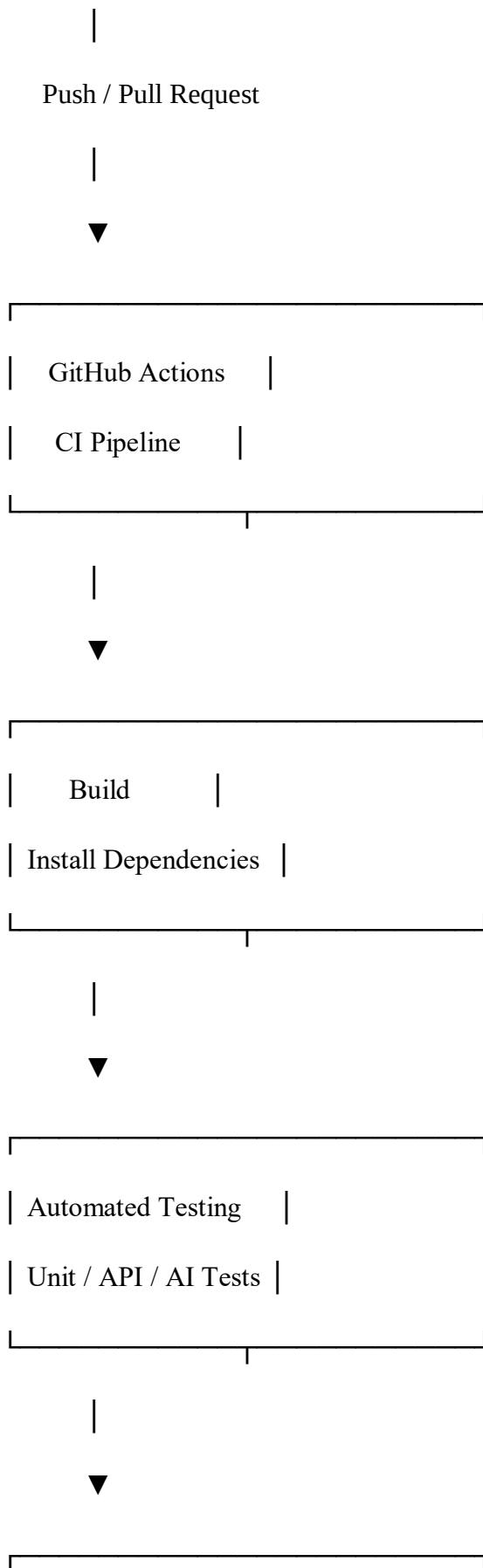
A rollback mechanism is required to recover from unsuccessful deployments. Every successful Docker image should be stored with a unique version or tag. If the latest production deployment introduces a serious problem, the previous stable image can be redeployed. Database changes should also be handled carefully because application rollback alone may not reverse database changes. Health checks can automatically identify whether the deployed application is functioning correctly. If a health check fails, the deployment can be stopped and the previous stable version restored. This reduces system downtime and protects users from unstable releases.

7. Prepare a CI/CD Pipeline Diagram and Explain the Workflow

7.1 CI/CD Pipeline Diagram

The following diagram represents the complete CI/CD workflow for the AI-Based Air Quality Monitoring and Prediction System:





| Quality & Security |

| Analysis |

|

|

Tests Passed

|



|

| Docker Build |

| Create Image |

|

|



|

| Container Registry |

| Store Image |

|

|



|

| Staging Environment |

| Integration Testing |

|

Approval

▼

Production Deployment

Docker Containers

▼

Monitoring & Alerts

Failure?

/ \

Yes No

▼ ▼

Rollback

Continue

Previous

Monitoring

Version



7.2 Explain the Workflow from Code Commit to Deployment

The workflow starts when a developer completes a change and pushes the code to the GitHub repository. A GitHub Actions workflow is automatically triggered by the commit or pull request. The pipeline first checks out the source code and installs the required dependencies. The application is then built and automated tests are executed. If the tests fail, the pipeline stops and developers receive a failure notification. If the tests pass, code quality and security checks are performed. A Docker image is then created and stored in a container registry. The image is deployed to the staging environment for additional testing and validation. After successful approval, the same tested image is deployed to production. The production environment is monitored after deployment. If a critical problem occurs, the pipeline can roll back to the previously verified version.

Overall Pipeline

Code Commit → Build → Test → Quality Check → Security Check → Docker Build → Staging → Approval → Production → Monitoring → Rollback if Required

Deliverable: Submit a CI/CD Pipeline Design document containing the pipeline architecture/diagram, stages, tools, configuration/workflow, testing strategy, deployment process, and justification based on the assigned problem statement.

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Requirement & Pipeline Analysis(15%)	Clearly analyzes the assigned problem and identifies comprehensive CI/CD requirements	Identifies most relevant requirements	Identifies basic requirements with some gaps	Requirements are unclear or incomplete
Pipeline Architecture & Workflow(25 %)	Designs a complete, logical pipeline covering source, build, test, quality check, package, and deployment stages	Pipeline covers most major stages with minor gaps	Basic pipeline is designed but several stages are missing	Pipeline is incomplete or poorly structured
Tools & Technology Selection(20%)	Selects appropriate tools for each stage and provides clear justification	Appropriate tools selected with reasonable justification	Tools are identified but justification is limited	Tools are inappropriate or not clearly identified
Automated Testing & Quality Integration(15 %)	Effectively integrates automated testing and code-quality/security checks into the pipeline	Includes major testing and quality checks	Includes basic testing with limited integration	Testing/quality checks are missing or inappropriate
Deployment, Security & Rollback Strategy(15%)	Clearly designs deployment environments, security practices, monitoring, and rollback strategy	Covers deployment and most security/rollback aspects	Basic deployment strategy with limited security/rollback details	Deployment strategy is incomplete or lacks security considerations
Pipeline Diagram & Documentation (10%)	Clear, accurate, professional diagram with excellent explanation and documentation	Well-presented diagram and documentation with minor issues	Adequate diagram/documentation but lacks clarity	Poorly presented, incomplete, or difficult to understand

Criteria	Mark
Requirement & Pipeline Analysis(15%)	/5
Pipeline Architecture & Workflow(25%)	/4
Tools & Technology Selection(20%)	/4
Automated Testing & Quality Integration(15%)	/4
Deployment, Security & Rollback Strategy(15%)	/4
Pipeline Diagram & Documentation(10%)	/4
TOTAL	/25